

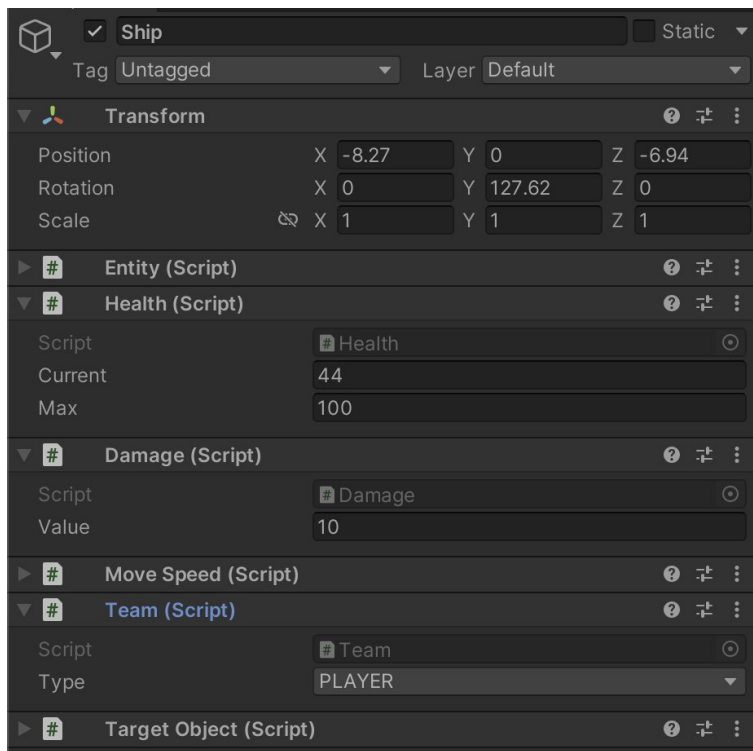
Техническое задание

Реализовать систему загрузки и сохранения игровых объектов на сервер для RTS игры



Описание игры:

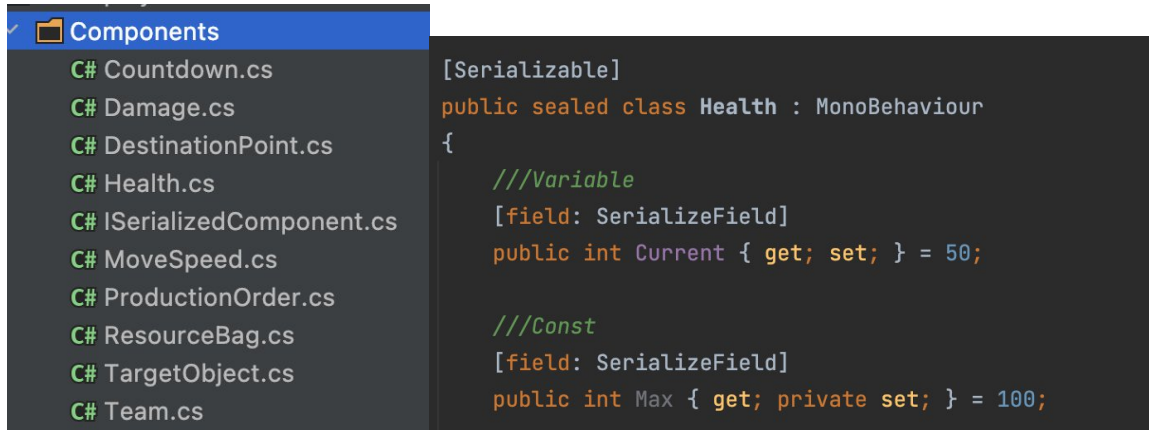
На игровом поле расположены игровые объекты: юниты, строения и ресурсы. Каждый юнит имеет различные компоненты (монобеги): здоровья, скорости, урона и т.д., которые отражают его характеристики



Сохранение и загрузка игровых объектов:

Необходимо сохранять и загружать всех юнитов, которые размещены на сцене со всеми их компонентами, а именно:

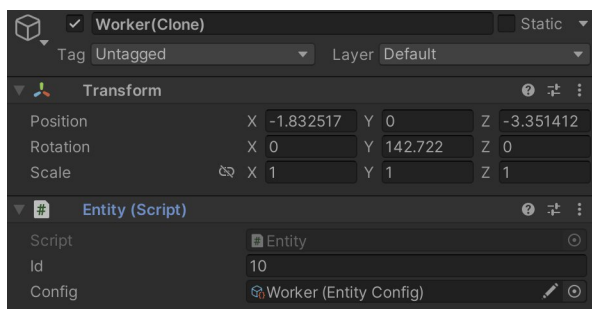
- Сами игровые сущности, их кол-во может меняться на сцене
- Позиция, поворот, id, имя сущности
- Компоненты, у которых есть свойства с комментарием **Variable**. Если над полем стоит комментарий **Const**, то этот параметр сохранять не нужно

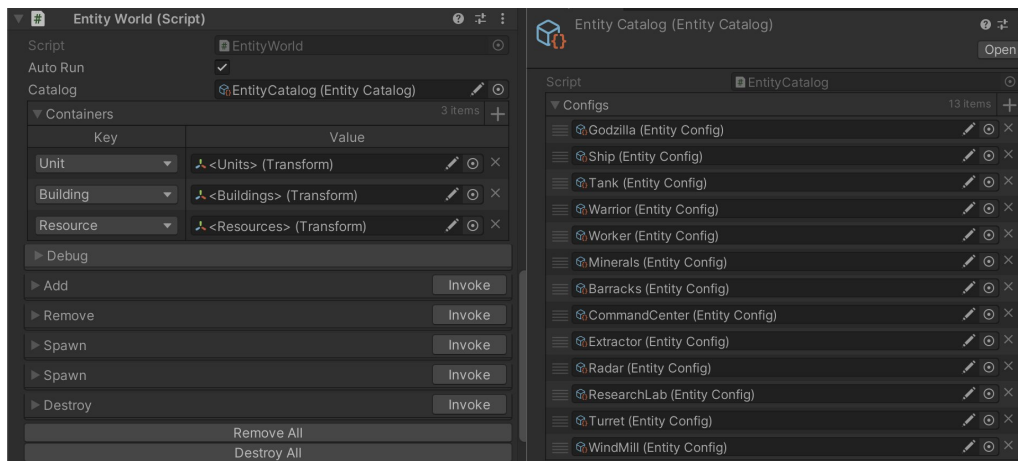


Сами компоненты можно расширять, но исходную структуру свойств и полей компонентов трогать нельзя!

Для работы с игровыми объектами есть модуль Entities, в котором есть:

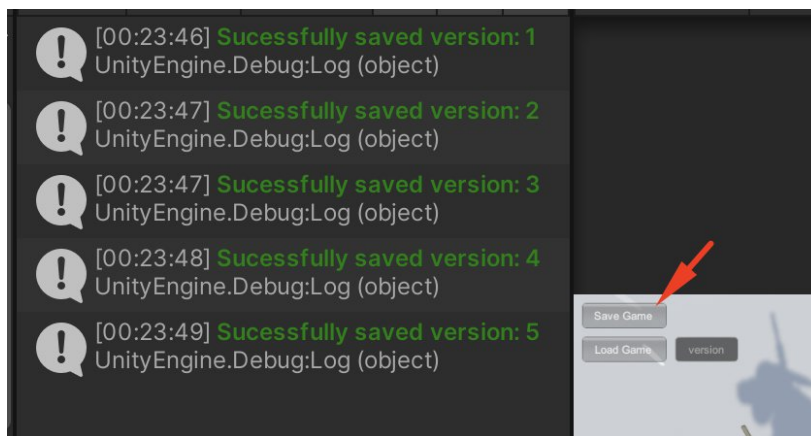
- Класс Entity — отвечает за основную информацию игрового объекта
- Класс EntityCatalog — представляет все возможные игровые объекты в виде конфигов
- Класс EntityWorld — отвечает за хранение активных игровых объектов на сцене, умеет создавать и удалять игровые объекты по имени, получать объекты по id.



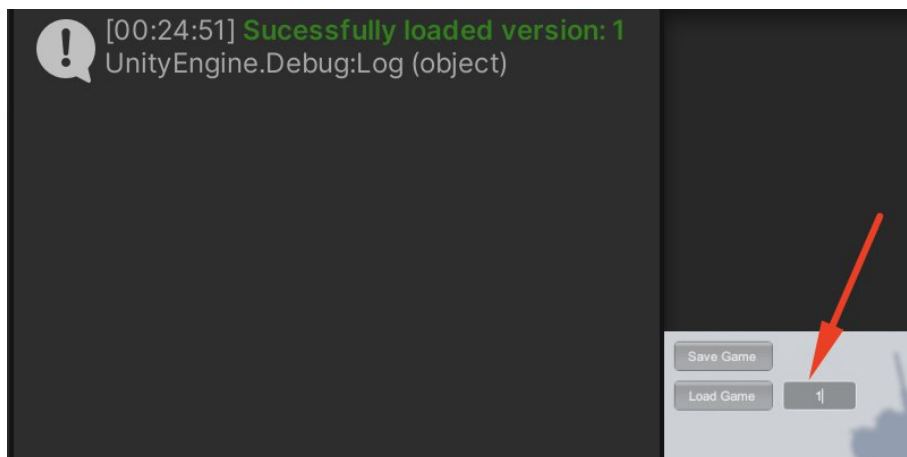


Контроль версий игры

Когда игрок сохраняет состояние игры, то этому сохранению автоматически присваивается номер версии и увеличивается на 1.



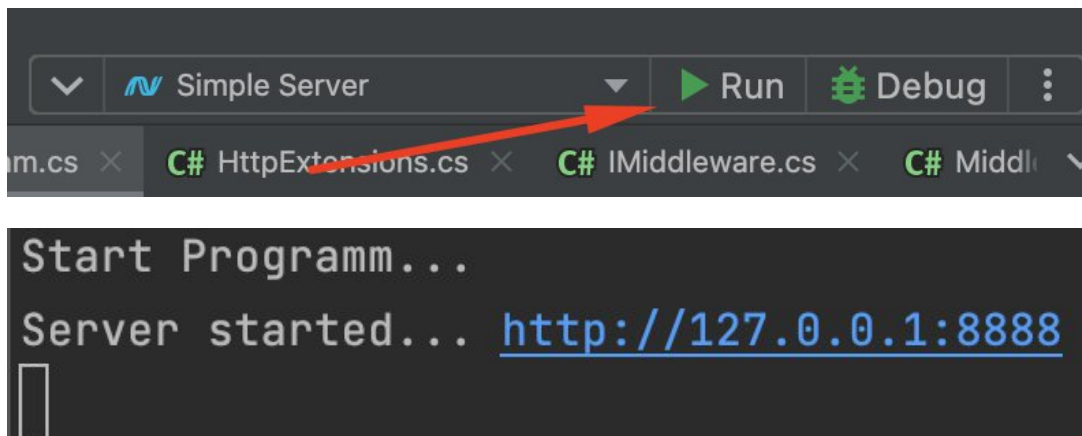
По этому номеру версии игрок может загрузить игру с определенным состоянием



Последний номер версии можно сохранять локально в PlayerPrefs

Работа с сервером

Для отправки запросов сохранения и загрузки в папке с домашним заданием есть архив HttpServer.zip, в котором уже написан сервер на языке C#. Этот сервер также можно открыть через IDE Visual Studio или Rider и запустить, нажав кнопку Run



Сервер работает на локальном хосте и слушает порт 8888.

Принимает следующие запросы:

1. **Save**
 - Метод: Put
 - Адрес: <http://127.0.0.1:8888/save?version={x}>
 - Параметры: номер версии x
 - Тело: состояние игры в формате текста
 - Ответы:
 - o 200 — Ok
 - o 400 — BadRequest
2. **Load**
 - Метод: Get
 - Адрес: <http://127.0.0.1:8888/load?version={x}>
 - Параметры: номер версии x
 - Ответы:
 - o 200 — данные игры в виде текста
 - o 400 — сообщение с ошибкой

Для отладки сервера можно использовать программу [Postman](#)

Интерфейс игры



Для отладки загрузки и сохранения необходимо написать ControlsPresenter

```
public sealed class ControlsPresenter : IControlsPresenter
{
    public void Save(Action<bool, int> callback)
    {
        throw new NotImplementedException();
    }

    public void Load(string versionText, Action<bool, int> callback)
    {
        throw new NotImplementedException();
    }
}
```

Итого:

- Сделать загрузку и сохранение всех игровых объектов на сцене и их компонентов
- Сделать версионирование состояний игры локально
- Реализовать запросы загрузки и сохранения на сервер
- Реализовать ControlsPresenter для UI

Дополнительные задачи со звездочкой:

- * Реализовать AES шифрование данных при отправки на сервер
- ** Реализовать сохранение версий локально и синхронизацию

Критерии оценки

- Паттерн сериализации и десериализации юнитов
- Версионирование игры
- Взаимодействие с сервером
- Реализация ControlsPresenter
- Соблюдение ООП, SOLID, GRASP
- Применение GoF паттернов
- Читаемый код
- Использование асинхронного программирования

Разрешается использовать следующие технологии

- UniRx, R3, UniTask, Awaitable, Tasks/ValueTasks, LINQ, UnityWebRequest